NAME : R KEERTHANA
REG NO : 411625104031
DEPARTMENT : B.E CSE
SUBJECT : PYTHON
BATCH : 3

UNIT : 2 PROJECT

TITLE : CODE HUNTER GAME

Aim:

To develop a Code Hunter Game using Python where the computer generates a random 4-digit secret code (without repeating digits) and allows the user to guess the code within a limited number of attempts while calculating the score based on remaining attempts.

Procedure:

Step 1:

Import the required module (random) to generate a random secret code.

Step 2:

Create a tuple containing digits from 1 to 9.

Step 3:

Create a dictionary to store player information such as:

- Name
- Attempts
- Maximum attempts
- Status
- Score

Step 4:

Create the required functions:

- generate_code() to generate a 4-digit secret code without repetition.

- is_valid_guess() to validate user input.
- evaluate_guess() to check exact and near matches.
- calculate_score() to calculate score based on remaining attempts.

Step 5:

Inside the main game function:

- Generate the secret code.
- Initialize an empty set to store guessed combinations.
- Display the game instructions.
- Set the maximum attempts to 8.

Step 6:

Use a while loop to allow the user to guess until:

- The user cracks the code, or
- The maximum attempts are reached.

If the guess is correct:

- Display a success message.
- Calculate the score.
- Exit the loop.

Step 7:

If the guess is incorrect:

- Display the number of exact and near matches.
- Decrease the number of attempts.
- Display remaining attempts.

Step 8:

If the user fails to guess within the given attempts:

- Display "Access Denied".
- Show the correct secret code.

Step 9:

Finally, display:

- Player name
- Status
- Attempts used
- Final score

CODE

```
import random


DIGITS = (1, 2, 3, 4, 5, 6, 7, 8, 9)


player = {
    "name": input("Enter your name: "),
    "attempts": 0,
    "max_attempts": 8,
    "status": "Playing",
    "score": 0
}


def generate_code():
    return tuple(random.sample(DIGITS, 4))



def is_valid_guess(guess):
    if len(guess) != 4:
        return False
    if not guess.isdigit():
        return False
    if len(set(guess)) != 4:
```

```
        return False
    if any(int(d) not in DIGITS for d in guess):
        return False
    return True


def evaluate_guess(secret, guess):
    exact = 0
    near = 0

    for i in range(4):
        if guess[i] == secret[i]:
            exact += 1
        elif guess[i] in secret:
            near += 1

    return exact, near


def calculate_score(player):
    remaining = player["max_attempts"] - player["attempts"]
    player["score"] = remaining * 10


def start_game():
    secret_code = generate_code()
    guessed_set = set()
```

```
print("\nCODE HUNTER")
print("Crack the 4-digit code (digits 1-9, no repeats)")
print("You have 8 attempts\n")

while player["attempts"] < player["max_attempts"]:

    guess_input = input("Enter 4 unique digits: ")

    if not is_valid_guess(guess_input):
        print("Invalid input. Use 4 different digits from 1-9.\n")
        continue

    guess = tuple(int(d) for d in guess_input)

    if guess in guessed_set:
        print("You already tried this combination.\n")
        continue

    guessed_set.add(guess)
    player["attempts"] += 1

    exact, near = evaluate_guess(secret_code, guess)

    if exact == 4:
        player["status"] = "Code Cracked"
        calculate_score(player)
        print("\nAccess Granted")
```

```
            break
        else:
            print("Exact:", exact, "| Near:", near)
            print("Attempts left:", player["max_attempts"] - player["attempts"], "\n")

    if player["status"] != "Code Cracked":
        print("\nAccess Denied")
        print("Secret Code was:", secret_code)

    print("\nPlayer:", player["name"])
    print("Status:", player["status"])
    print("Attempts Used:", player["attempts"])
    print("Score:", player["score"])
start_game()
```

OUTPUT:

The program prompts the user to enter their name.
The system generates a random 4-digit secret code without repeating digits.
The user is given 8 attempts to guess the code.
After each guess, the program displays the number of exact and near matches.
If the user guesses correctly, the program calculates and displays the score.
If the user fails, the correct code is displayed.

Result:

The Code Hunter Game program was successfully executed.
The program allows the user to guess a randomly generated 4-digit secret code within 8 attempts.
It correctly validates input, checks exact and near matches, and calculates the final score based on remaining attempts.